

Lezione 4: Processi



Obiettivi

- Nozione di processo
- Caratteristiche dei processi (schedulazione, creazione, terminazione, comunicazione, etc.)
- Comunicazione interprocesso
- Comunicazione nei sistemi client-server

Definizione di Processo

- Un SO esegue programmi di varia natura:
 - Sistemi batch – **job**
 - Sistemi time-shared – **programmi utente** o **task**
 - I libri di testo usano **job** e **processo** in modo quasi intercambiabile

- **Processo** – è un programma in esecuzione
 - Un programma è un'entità *passiva* archiviata su disco (**file eseguibile**), un processo è *attivo (in esecuzione)*
 - Unità di attività computazionale coerente di un moderno SO
 - Esecuzione sequenziale
 - Un processo richiede risorse (CPU, memoria, file, dispositivi I/O) per l'esecuzione di un compito (task)
 - Un programma può chiamare molti processi
 - I sistemi sono il risultato dell'esecuzione di più processi concorrenti
 - ▶ processi di sistema e processi utente

Definizione di Processo

- **Processo** – è un programma in esecuzione:
 - Stato corrente **program counter** e **registri del processore**
 - Layout di memoria

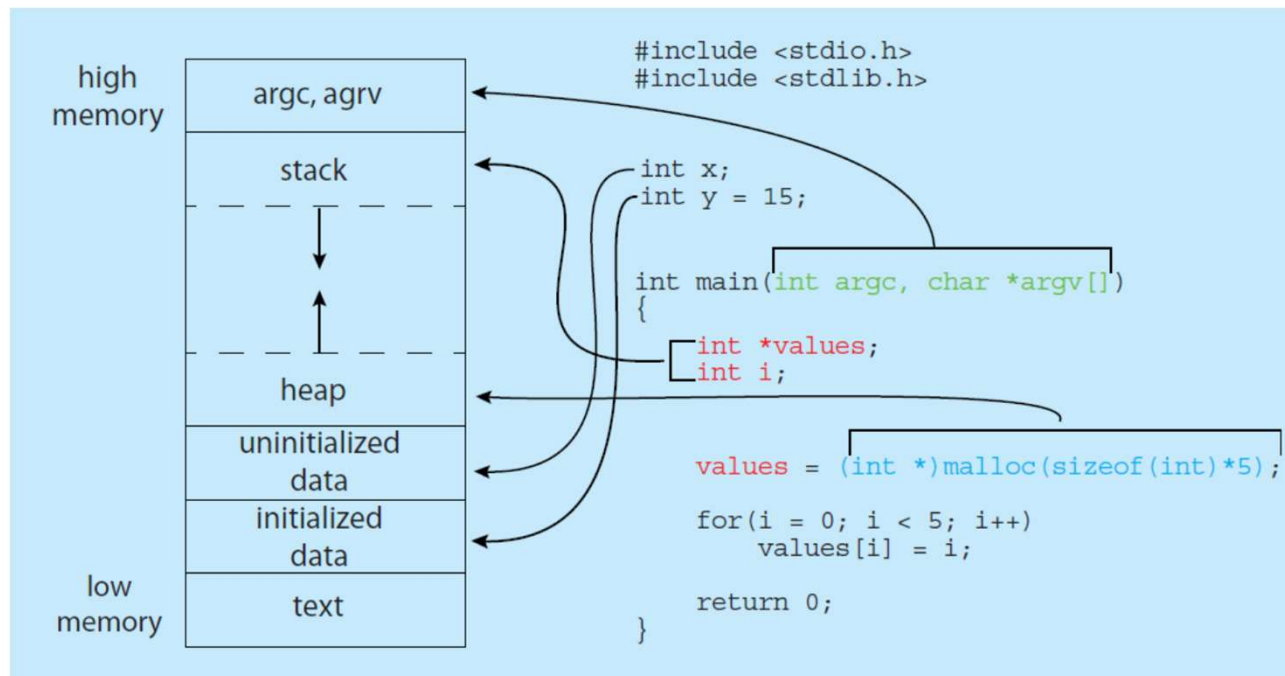
- Layout di memoria:
 - Il codice del programma, chiamata **sezione testo**
 - **Sezione Dati** contenente le variabili globali
 - **Stack** contenente i dati temporanei
 - ▶ Parametri di funzione, indirizzi di ritorno, variabili locali
 - **Heap** contenente memoria dinamicamente allocata a run time durante l'esecuzione di un task

Processo Allocato in Memoria

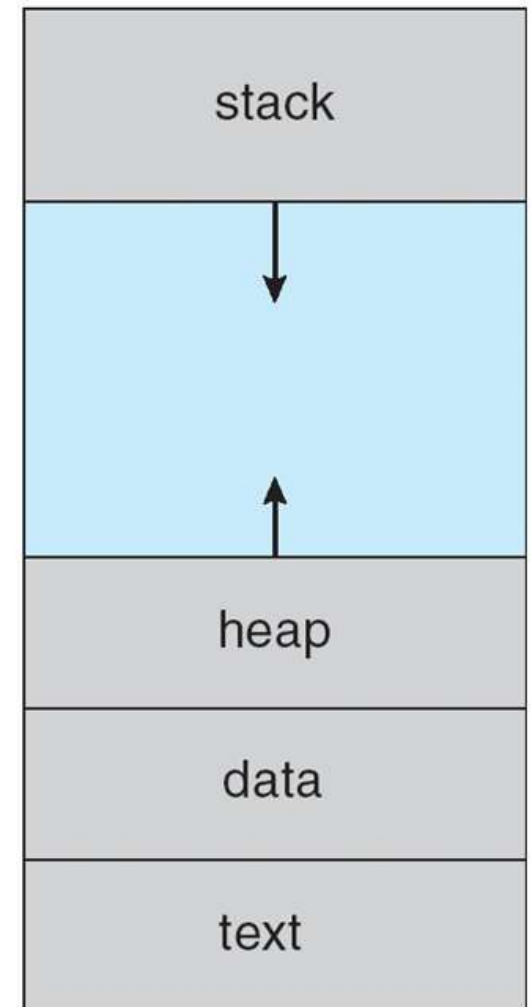
□ **Processo** – è un programma in esecuzione:

□ Layout di memoria:

- Text e Data dimensione fissata
- Stack e Heap variabili a run time



max

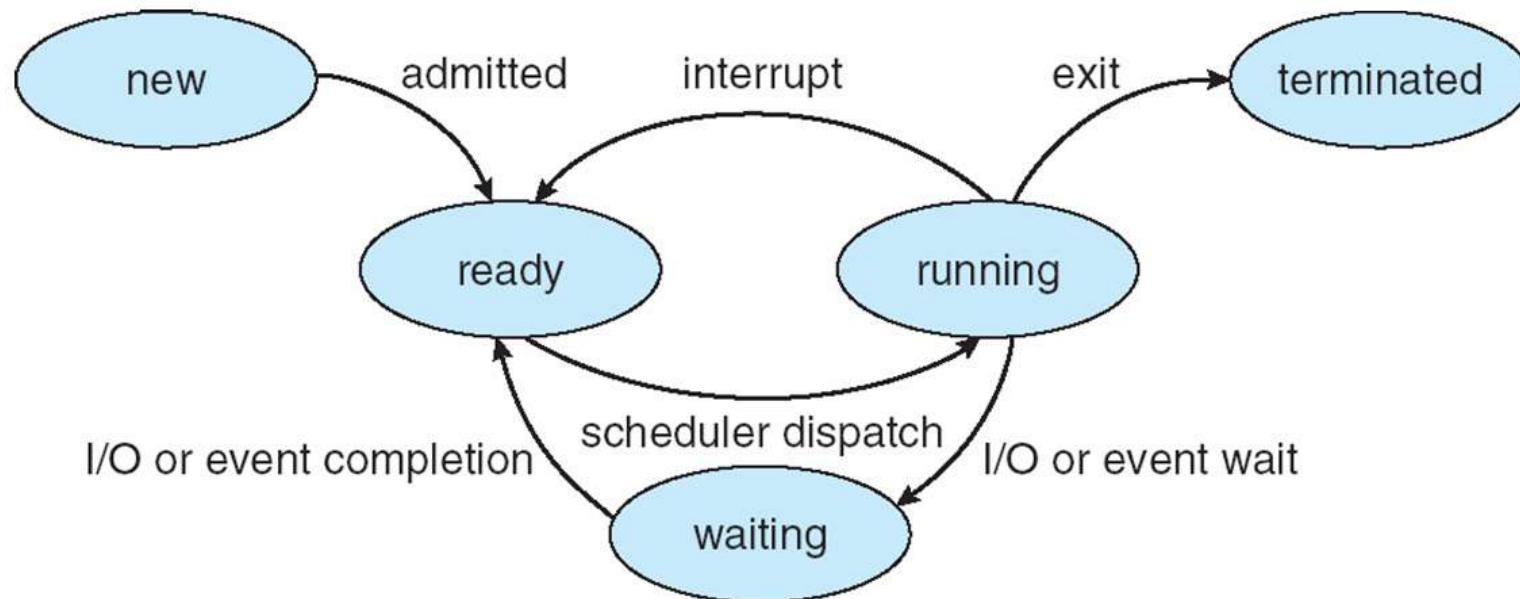


0

Non initialized data anche indicate come block started by symbol (bss)

Stato di un Processo

- Durante l'esecuzione un processo può cambiare il suo **stato**
 - **new**: il processo è stato creato
 - **running**: esecuzione delle istruzioni del processo
 - **waiting**: il processo è in attesa di un evento per proseguire (es. I/O)
 - **ready**: il processo è in attesa di essere assegnato a processore
 - **terminated**: il processo ha finite l'esecuzione



Descrittore di Processo

- Il Sistema Operativo mantiene una **tabella dei processi**
- Per ogni processo una entry (**Task Control Block**)
 - **Stato del processo** - running, waiting, etc
 - **IDs del processo** - identificativi di processo
 - **Program counter** - locazione prossima istruzione
 - **Registri CPU** - contenuto registri CPU
 - **Scheduling di CPU** - priorità, parametri di scheduling
 - **Memoria** - memoria allocata per il processo (es. base and limit registers, tabella delle pagine, etc.)
 - **Contabilità** - CPU usata, tempo clock dallo start, numero di processo, etc.
 - **Stato I/O** - dispositivi I/O allocati, lista di file aperti, etc.

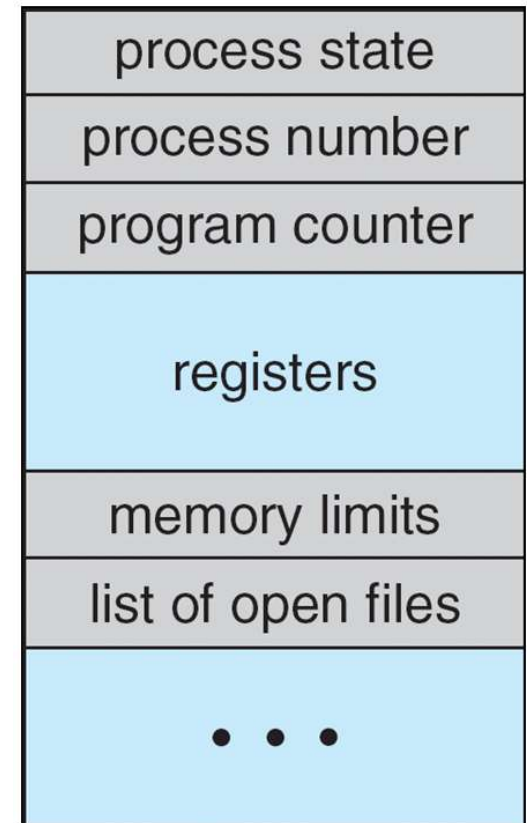


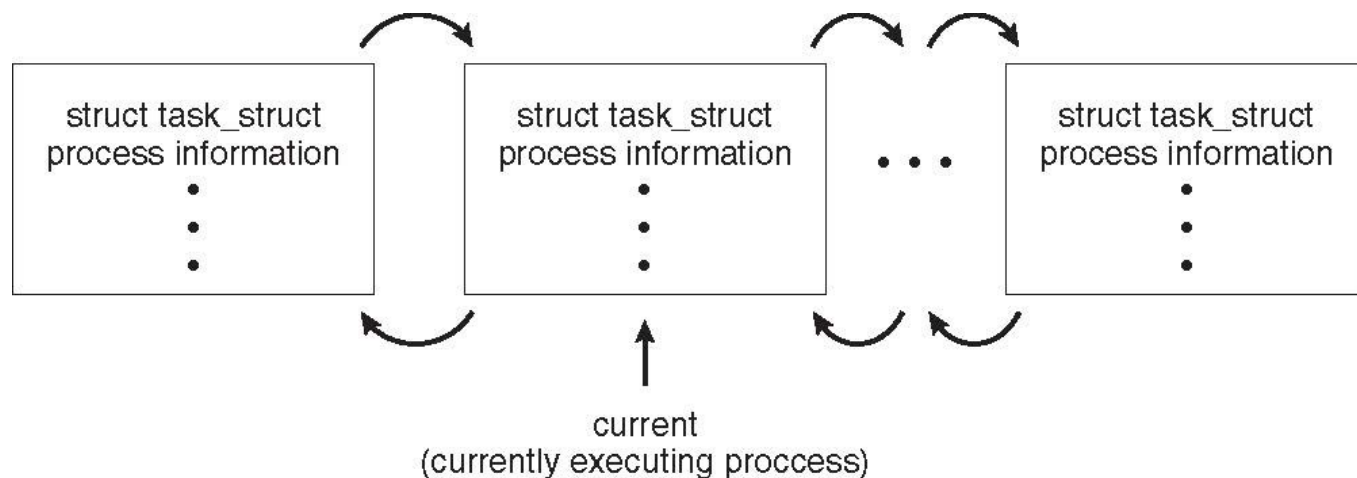
Tabella Processi in Linux

In Linux la PCB è rappresentata in C da una `task_struct` specificata nel codice sorgente del kernel (in `/linux/sched.h`)

Alcuni campi seguono:

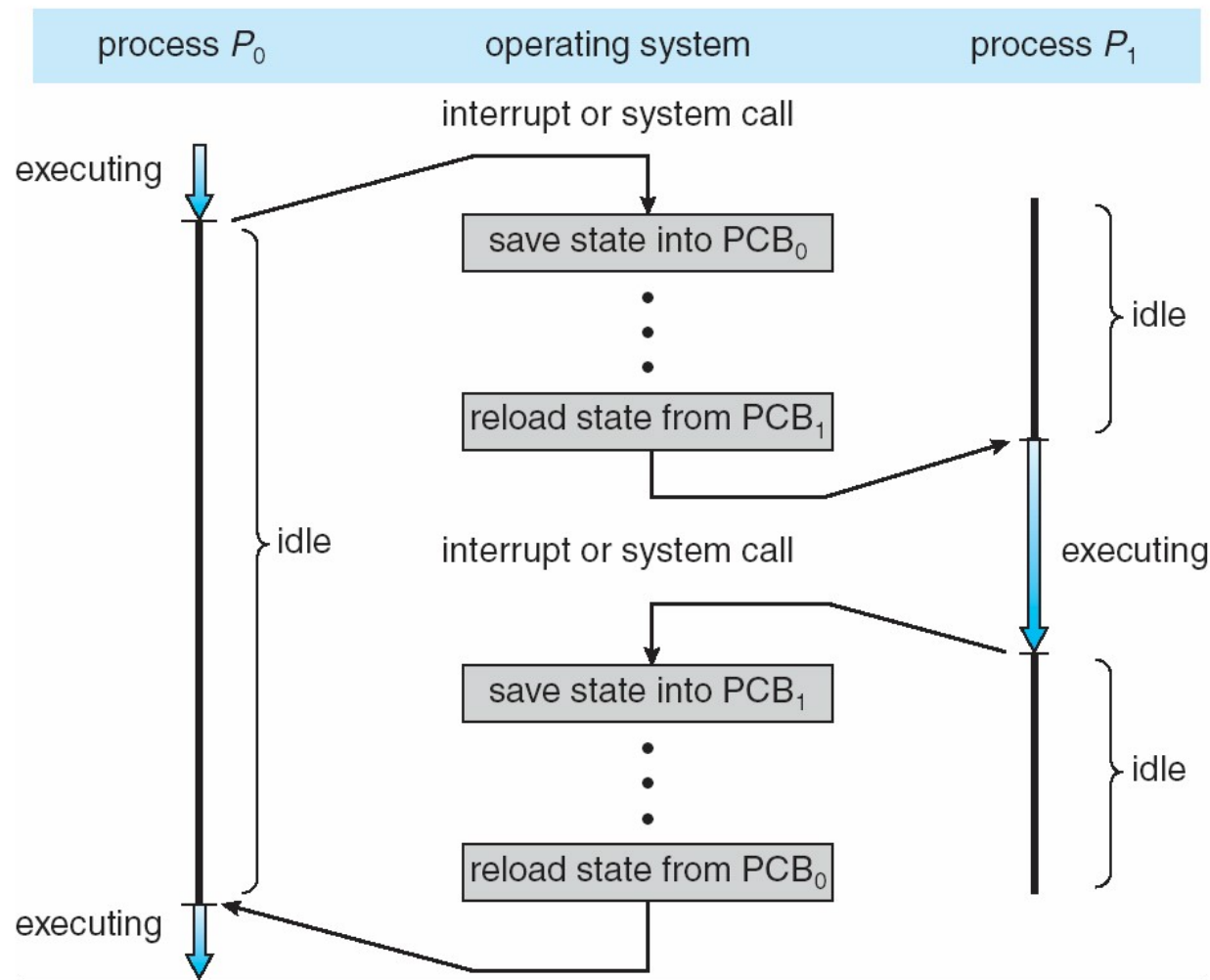
```
pid t_pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```

Nel kernel Linux i processi attivi sono rappresentati da una lista doppiamente linkata



Commutazione tra Processi

- La tabella dei processi supporta la **commutazione** tra processi
 - Lo stato dei processi salvato e ripristinato durante gli switch

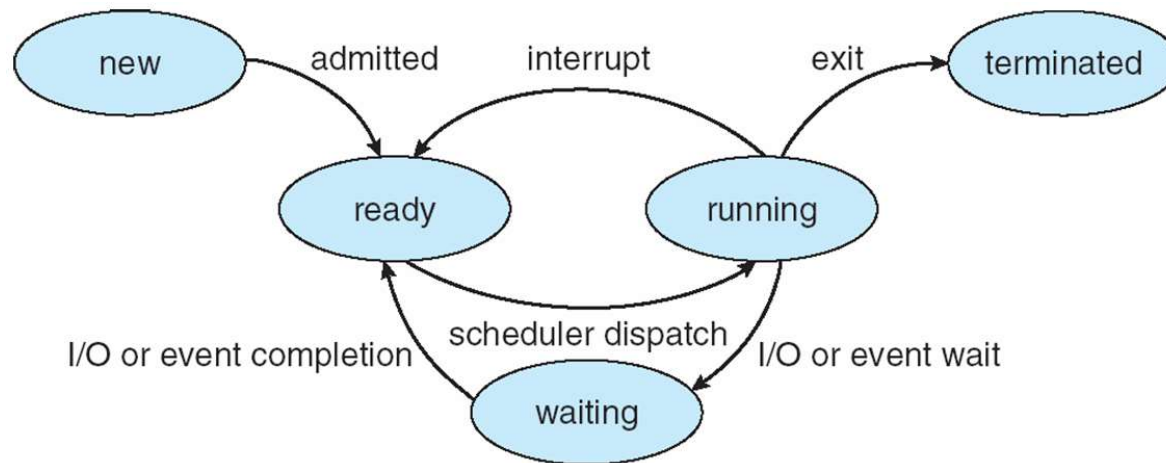


Processi e Thread

- I moderni Sistemi Operativi consentono ai processi di gestire l'esecuzione di più flussi di controllo (**thread**)
- I processi possono diramarsi in più sottounità di esecuzione
 - Più program counter per un processo
 - Multipli flussi di controllo -> threads
 - ▶ Condividono identità e risorse di processo ma hanno diverso stato esecutivo
 - Archiviazioni di ulteriori informazione per thread in PCB

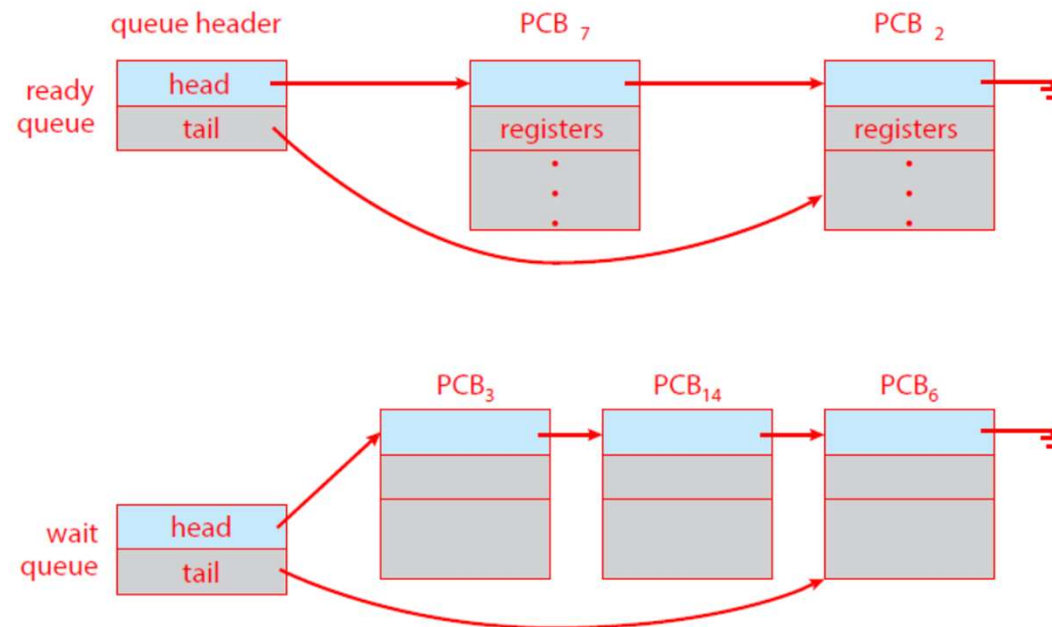
Schedulazione di Processi

- Massimizza l'uso della CPU, fa velocemente lo switch di processi nella CPU per il time sharing
- **Schedulatore dei processi** seleziona tra processi disponibili per la prossima esecuzione su CPU
- Mantiene la **coda di scheduling** dei processi
 - **Job queue** – insieme di tutti i processi nel sistema
 - **Ready queue** – insieme di tutti i processi in memoria principale, pronti (ready)
 - **Wait queue** – insieme dei processi in attesa (waiting)
 - **Device queues** – insieme di processi in attesa (waiting) di un dispositivo di I/O
 - I processi migrano tra le diverse code



Code Processi Ready e Waiting

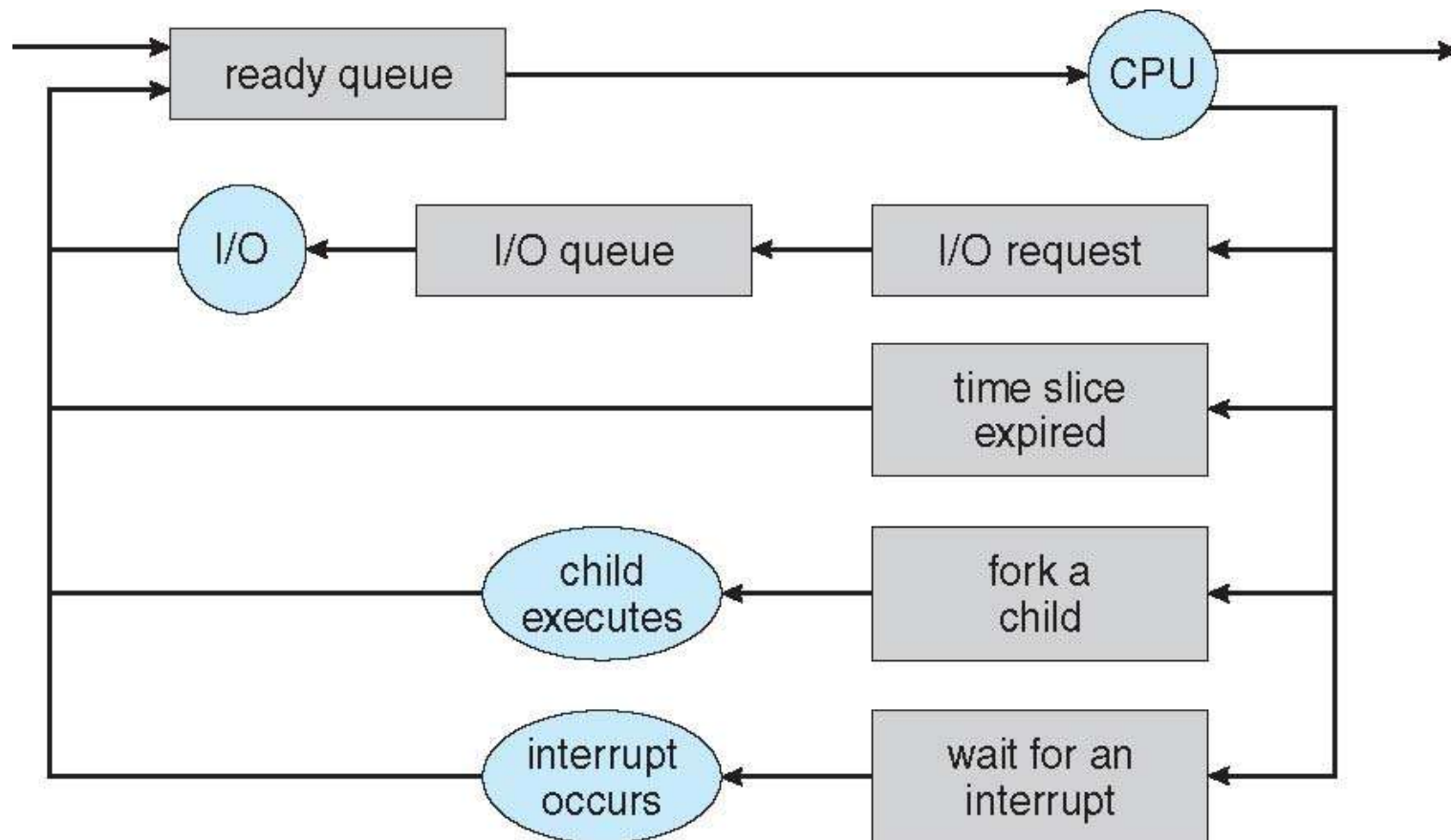
- Mantiene la **coda di scheduling** dei processi
 - **Job queue** – insieme di tutti i processi nel sistema
 - **Ready queue** – insieme di tutti i processi in memoria principale, pronti (ready) per l'esecuzione
 - **Wait queue** – insieme dei processi in attesa (waiting)
 - **Device queues** – insieme di processi in attesa (waiting) di un dispositivo di I/O
 - I processi migrano tra le diverse code



Rappresentazione Scheduling dei Processi

□ Diagramma di accodamento

- Un nuovo processo in coda ready attende la CPU
- Una volta allocata diversi eventi possono avvenire (I/O, fine tempo, fork, wait)



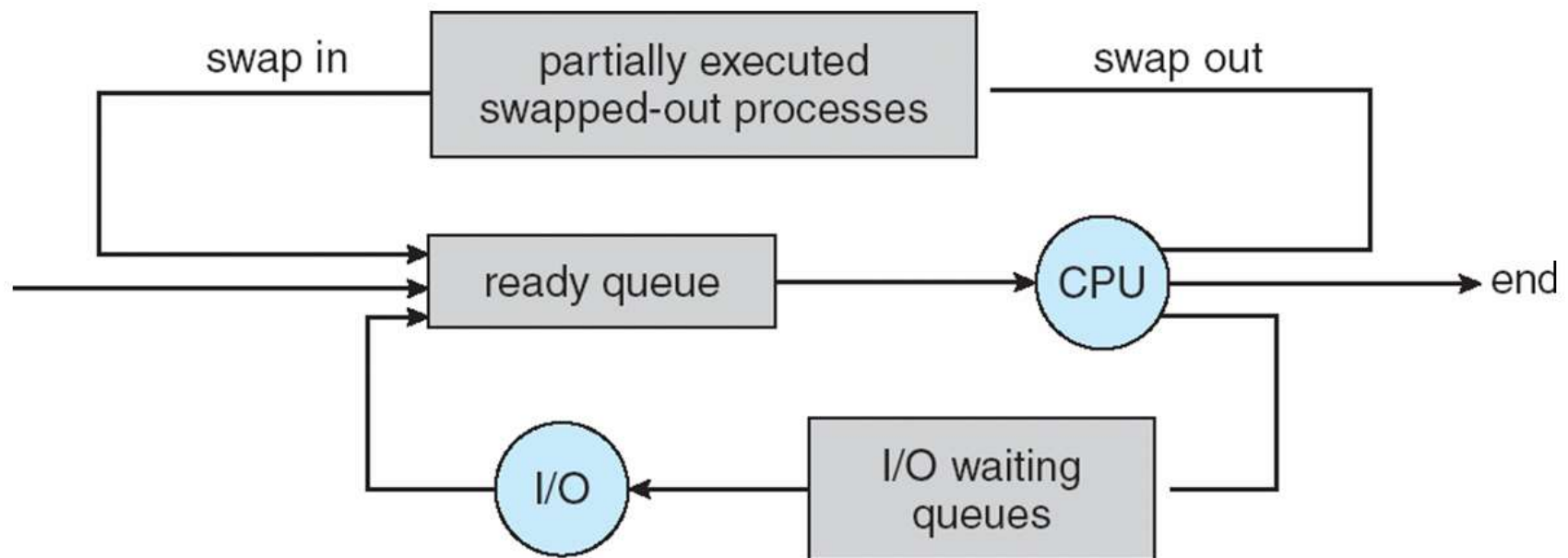
Scheduler

- **Short-term scheduler** (o **CPU scheduler**) – seleziona quale processo deve essere eseguito dalla CPU
 - A volte l'unico scheduler nel sistema
 - Lo short-term scheduler è invocato frequentemente (millisecondi) $\Rightarrow$ (deve essere veloce)

- **Long-term scheduler** (o **job scheduler**) – seleziona quale processo deve essere portato nella coda ready
 - Long-term scheduler non frequente (secondi, minuti) $\Rightarrow$ (può essere lento)
 - Il long-term scheduler controlla il **grado di multiprogrammazione**
 - I processi possono essere descritti come:
 - ▶ **I/O-bound process** – spende più tempo in operazioni I/O che in computazione, piccoli e brevi accessi in CPU
 - ▶ **CPU-bound process** – spende più tempo in computazione; pochi e lunghi accessi in CPU
 - ▶ Long-term scheduler cerca di trovare una buona combinazione di processi

Scheduling di Medio Termine

- **Medium-term scheduler** può essere aggiunto se il grado di multiprogrammazione deve decrescere (minor contesa per CPU)
 - Toglie processi dalla memoria, archivia su disco, riporta in memoria da disco per continuare l'esecuzione: **swapping**



Context Switch

- Al passaggio della CPU ad un altro processo il sistema deve **salvare lo stato** del vecchio processo e caricare lo **stato salvato** del nuovo processo attraverso un **context switch**
- **Context** (contesto) di processo rappresentato in PCB
- Il tempo di context-switch è un **overhead**
 - il sistema non fa lavoro “utile” durante lo switching
 - Più è complesso il sistema e la PCB più è lungo il context switch
- Il tempo dipende dal supporto hardware
 - Alcuni hardware forniscono registri multipli per CPU consentendo caricamenti di più context in una volta

Operazioni su Processi

- Il Sistema deve fornire meccanismi per:
 - Creazione di processi,
 - Terminazione di processi,
 - Comunicazione tra processi
 - Sincronizzazione tra processi
 - Altri meccanismi di gestione di processi (dettagliato dopo)

Creazione di Processi

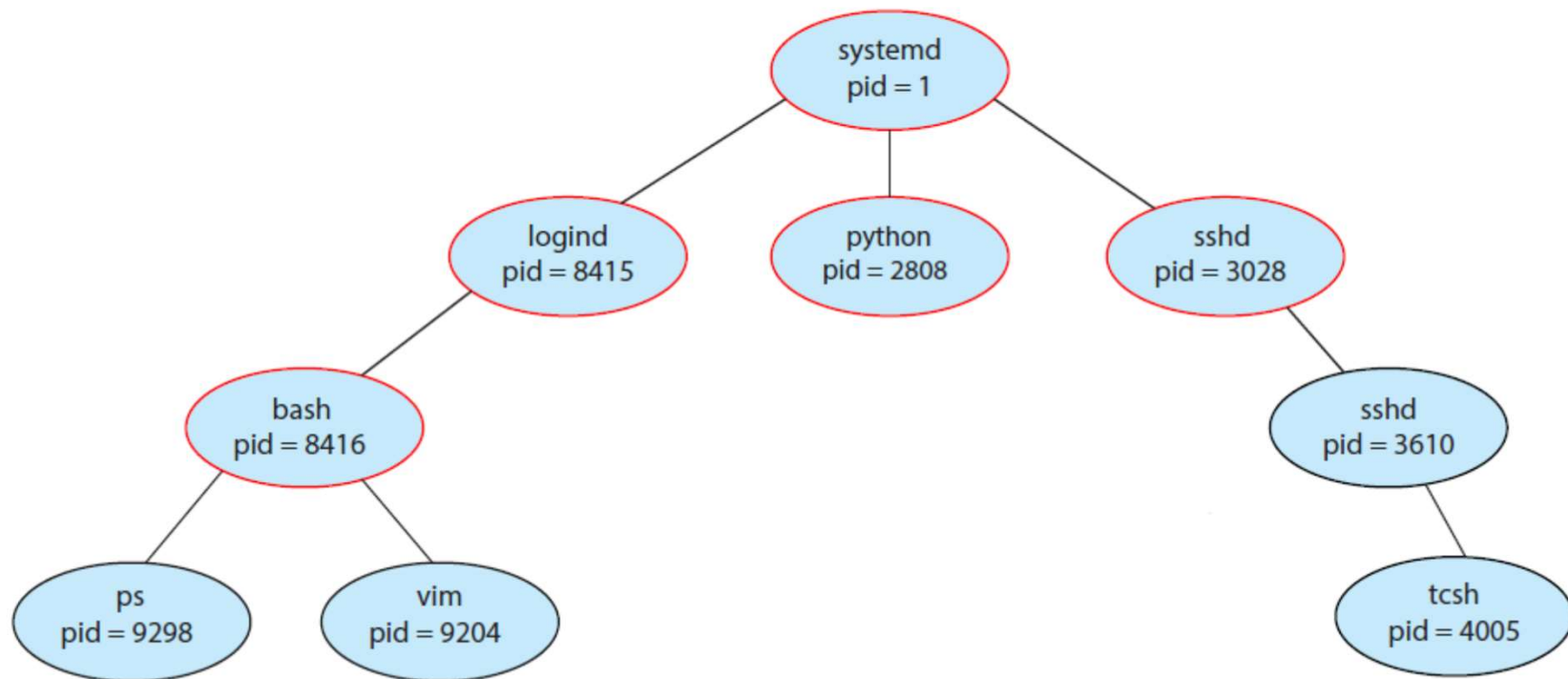
- Processo **padre** crea un processo **figlio**, che a sua volta crea altri processi formando un **albero** di processi
- Processi identificati e gestiti con **identificatori di processo (pid)**
- Condivisione di risorse (opzioni)
 - Padre e figlio condividono tutte le risorse
 - Figlio condivide un sottoinsieme delle risorse del padre
 - Padre e figlio non condividono alcuna risorsa
- Opzioni di esecuzione
 - Padre e figlio vengono eseguiti in concorrenza
 - Padre aspetta che il figlio finisca

Albero di Processi in Linux

System daemon fa l'init
Logind gestisce il log nel Sistema
L'utente lancia una shell bash, fa ps e vim

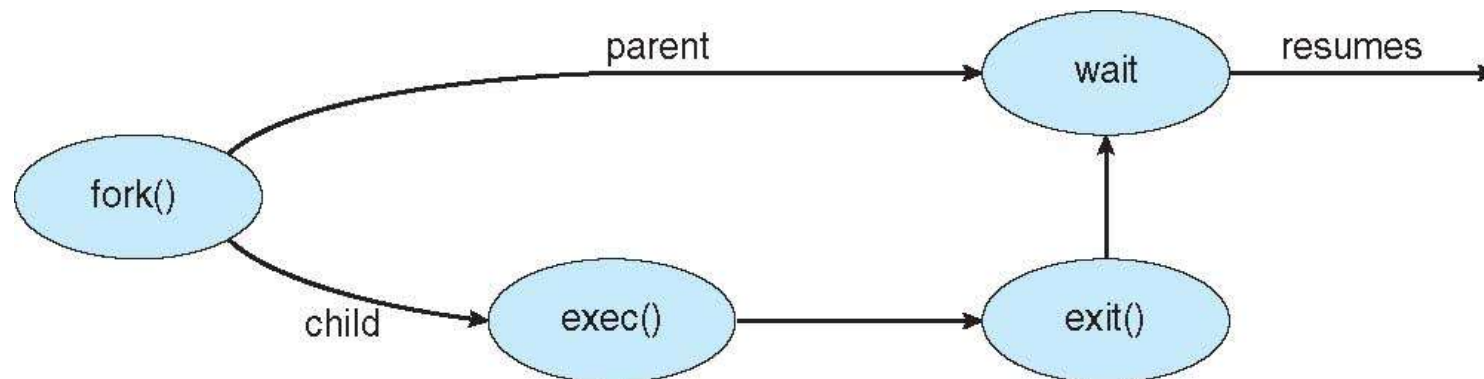
In UNIX il primo processo è init

Per vedere l'albero dei processi da bash: pstree



Creazione di Processi

- Diverse gestioni
 - Spazio memoria
 - ▶ Figlio duplica quello del padre
 - ▶ Figlio ha un programma caricato
 - Esecuzione:
 - ▶ Padre e figlio concorrenti o padre attende figlio
- Esempi UNIX
 - `fork()` system call crea nuovo processo
 - `exec()` system call usata dopo la `fork()` per sostituire lo spazio di memoria con un nuovo programma



Programma C con fork ed exec

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

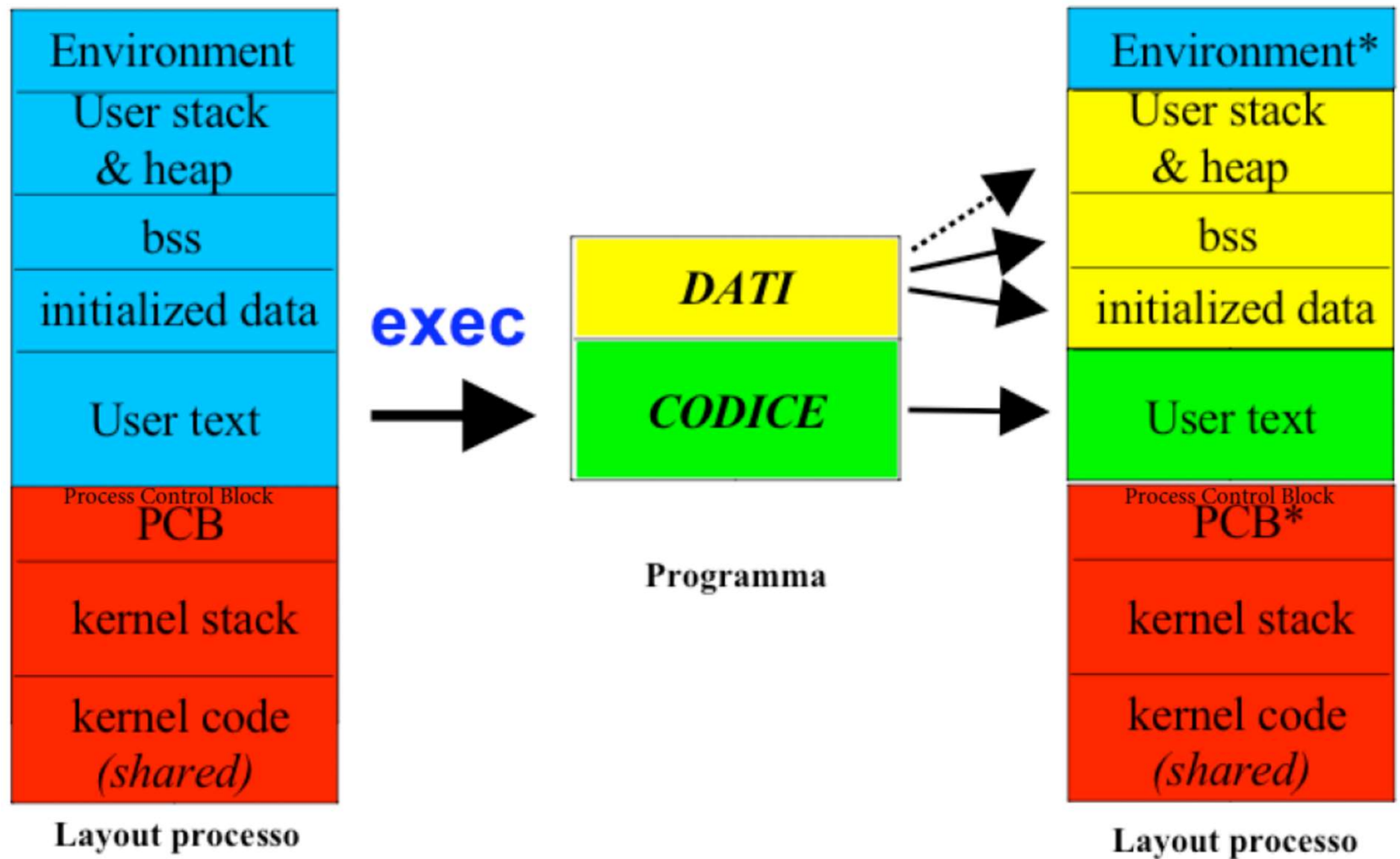
int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```

Programma C con fork ed exec



Creare Processo con Windows API

```
#include <stdio.h>
#include <windows.h>

int main(VOID)
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
        "C:\\WINDOWS\\system32\\mspaint.exe", /* command */
        NULL, /* don't inherit process handle */
        NULL, /* don't inherit thread handle */
        FALSE, /* disable handle inheritance */
        0, /* no creation flags */
        NULL, /* use parent's environment block */
        NULL, /* use parent's existing directory */
        &si,
        &pi))
    {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }

    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

Proprietà dei nuovi processi
struttura info, es. finestra e gestione I/O
struttura informazioni contiene handle e identificatori del nuovo processo

Si alloca memoria per queste strutture
Analogo di memset a zero in Windows

Application name e parametri
command-line
setting di default per la creazione del figlio
Si popolano si e pi

Attesa infinita del processo
indicato da pi.hProcess
(processo figlio)

Programma C con fork

```
int main(void) {  
    int i;  
  
    for (i=0; i<2 ;i++)  
        if (fork())>0 {  
            printf("Padre! %d\n", i);  
        } else {  
            printf("Figlio! %d\n", i);  
        }  
  
    sleep(10);  
    return 0;  
}
```

- Qual è l'output di questo programma?
- Quanti processi vengono creati?
- Di chi è figlio ciascun processo creato?

Programma C con fork

\$/a.out

padre 0

padre 1

figlio! 0

padre 1

figlio! 1

figlio! 1

Proc principale

padre 0

figlio! 0

padre1 figlio! 1

padre1 figlio! 1

Terminazione Processo

- I processi eseguono le ultime istruzioni e poi chiedono all'SO di uscire con la call di sistema **exit()**
 - Ritorna i dati di status data dal figlio al padre (via **wait()**)
 - Le risorse sono deallocate dall'SO

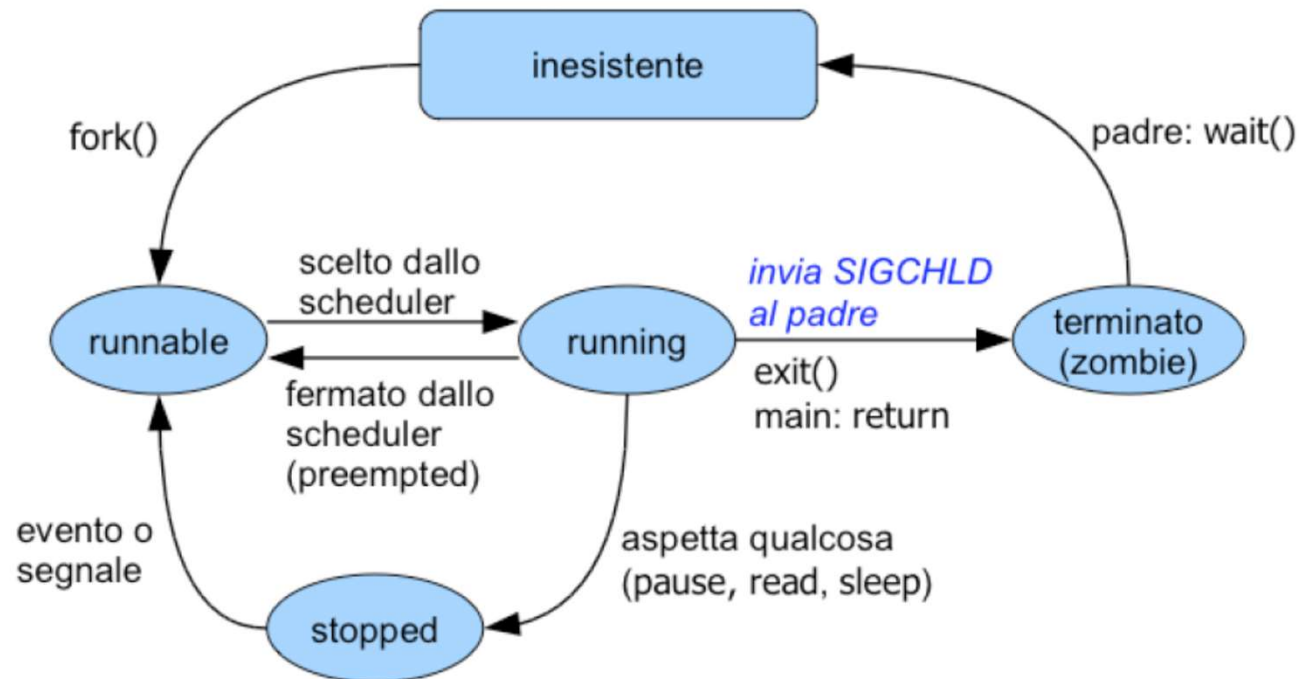
- Padri possono terminare le esecuzioni dei processi figli con la call **abort()** ad esempio perché:
 - Il figlio ha allocato risorse in eccesso
 - Il task del figlio non è più richiesto
 - Il genitore esce e non consente a un figlio di continuare

Terminazione Processo

- Alcuni SO non permettono ai figli di esistere se i padri hanno terminato
 - Se un processo termina tutti i figli terminano
 - **cascading termination** (tutta la gerarchia di figli termina)
 - la terminazione è gestita dal SO
- Il padre può aspettare la terminazione di un figlio con la call **wait()**
 - riceve informazioni di stato e il pid del processo terminato, es. Unix
pid = wait(&status);
- In Unix/Linux se non c'è padre in waiting il figlio diventa **zombie**
- Se padre termina prima del figlio senza invocare **wait** il figlio è **orphan**
 - In Unix adottato da init che chiama continuamente wait per accogliere lo status (in linux init è oggi systemd – System Daemon)

Terminazione Processo

□ In UNIX/Linux



Terminazione Processo

□ In UNIX/Linux

/ status.c: Fornisce un esempio dei diversi stati di uscita di un processo figlio */*

```
#include<sys/types.h> /* per il tipo pid_t */
#include <unistd.h> /* per la funzione fork */
#include<sys/wait.h> /* per la funzione waitpid */
#include <stdio.h> /* per le funz. printf, fgets e perror */
#include <stdlib.h> /* per la funzione exit */
#include <string.h> /* per la funzione strlen */
#include <errno.h> /* per la messaggistica di errore */

int main(void)
{
    pid_t pid;
    int status;

    if ( (pid = fork()) < 0)
        perror("fork"), exit(1);
    else if (pid == 0) /* primo figlio... */
        exit(7); /* che termina in modo normale */

    if (wait(&status) != pid) /* il genitore aspetta il figlio... */
        perror("wait"), exit(1);
    print_exit(status); /* e ne mostra lo stato di terminazione */
}
```

```
if ( (pid = fork()) < 0)
    perror("fork"), exit(1);
else if (pid == 0) /* secondo figlio... */
    abort(); /* che genera SIGABRT */
```

```
if (wait(&status) != pid) /* il genitore aspetta il 2ndo figlio... */
    perror("fork"), exit(1);
print_exit(status); /* e ne mostra lo stato di terminazione */
```

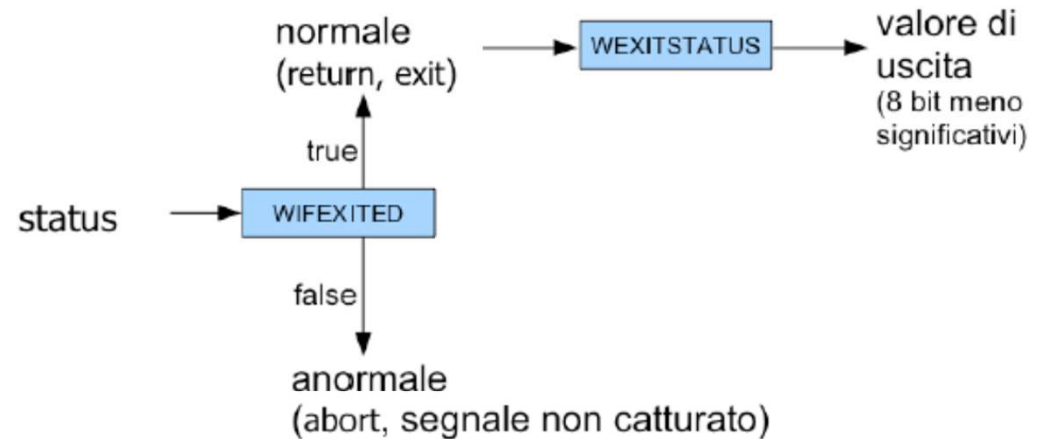
```
if ( (pid = fork()) < 0)
    perror("wait"), exit(1);
else if (pid == 0) /* terzo figlio.... */
    status /= 0; /* che divide per 0 e genera SIGFPE */
```

```
if (wait(&status) != pid) /* il genitore aspetta il terzo figlio... */
    perror("wait"), exit(1);
print_exit(status); /* e ne mostra lo stato di terminazione */
```

```
exit(0);
```

Terminazione Processo

- In UNIX/Linux
 - Stato di uscita



```
int status;
```

```
wait(&status);
```

```
if ( WIFEXITED(status) )
```

```
    printf("valore di uscita: %d\n", WEXITSTATUS(status));
```

```
else
```

```
    printf("terminazione anomala\n");
```

```
#define WIFEXITED(status) (((status) & 0x7F) == 0)
```

E.g., `status = 10000000` with `0x7F = 01111111`

Architetture Multiprocesso – Chrome Browser

- ❑ Molti web browser giravano come un singolo processo (alcuni ancora)
 - ❑ Se un website causa problemi il browser può crashare o stallare
- ❑ Google Chrome Browser è multiprocesso con differenti tipi di processo:
 - ❑ **Browser** che gestisce l'interfaccia utente, il disco e la rete
 - ❑ **Renderer** che rende graficamente le web pages, gestisce HTML, Javascript. Per ogni sito web aperto è lanciato un nuovo renderer
 - ❑ **Plug-in** per ogni tipo di plug-in

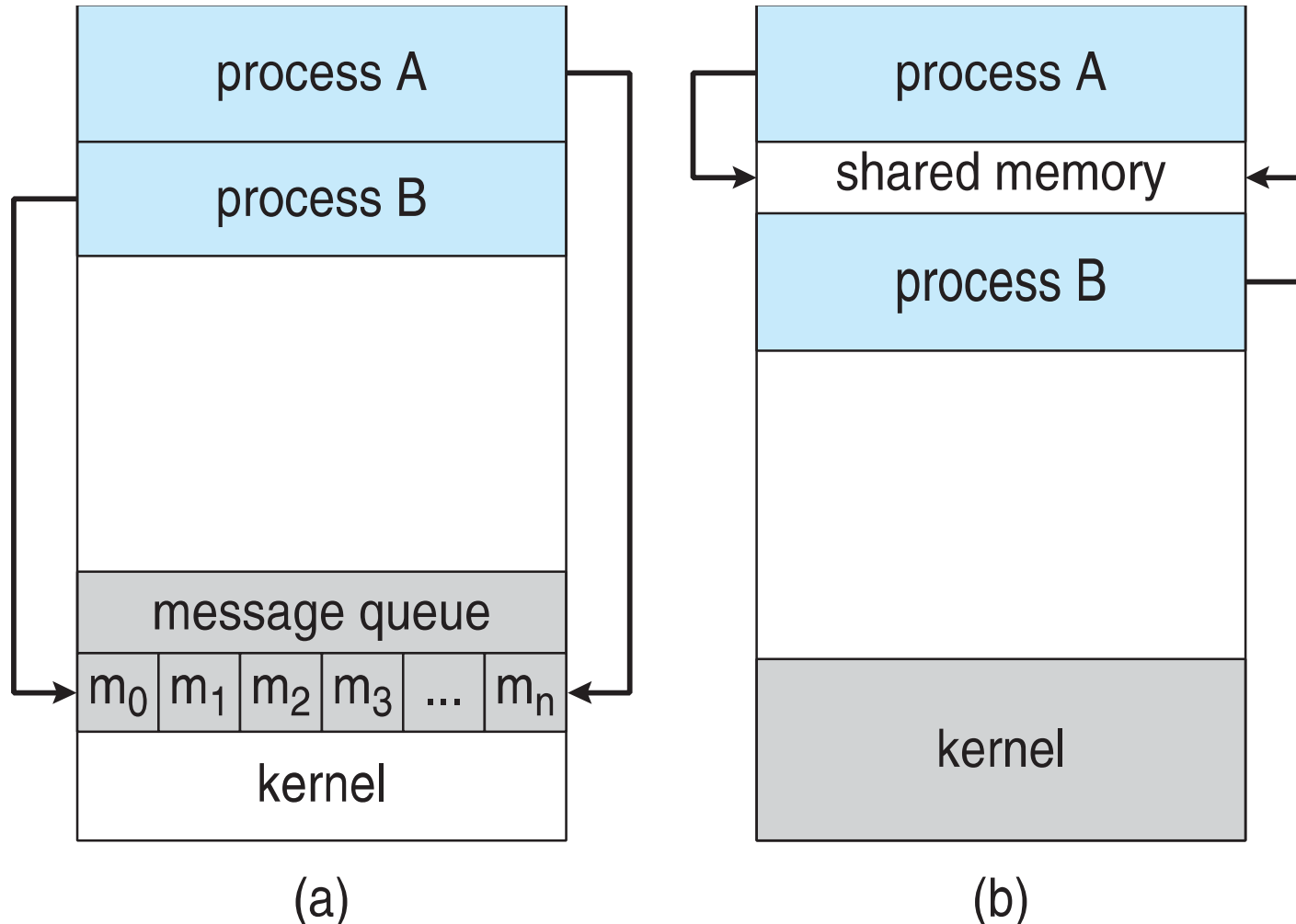


Comunicazione Interprocesso

- Processi possono essere *independenti* o *cooperati*
- Processi cooperanti si influenzano scambiandosi dati
 - Diverse sono le ragioni per la cooperazione tra processi:
 - ▶ Condivisione di dati (accesso a dati condivisi)
 - ▶ Velocità computazionale (sottoprocessi lavorano in parallelo)
 - ▶ Modularità (divisione funzionale dei compiti)
- Processi cooperanti hanno bisogno di **interprocess communication (IPC)**
 - Due modelli principali di IPC
 - ▶ **Shared memory**
 - ▶ **Message passing**

Modelli di Comunicazione

(a) Message passing. (b) shared memory.



Scambio di poca informazione, tutti gli scambi passano per il Kernel

Più veloce (kernel alloca la zona condivisa), più semplice accesso, problema coordinazione tra processi

Memoria Condivisa

- Comunicazione su regione di **shared memory**
- Tipicamente creata nello spazio di indirizzamento del processo che la crea (segmento di shared memory)
- Gli altri processi devono accedere a questo spazio che generalmente è proibito (va rilassata la restrizione)
- I processi leggono e scrivono su questo spazio che è gestito dai processi (modalità utente)
 - La coordinazione di lettura e scrittura è sotto la responsabilità dei processi coinvolti, il SO lascia fare

Problema Produttore-Consumatore

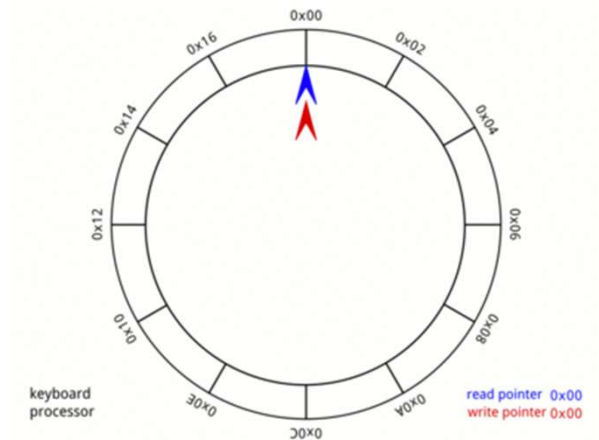
- Si introduce il problema del **Produttore-Consumatore**
- Paradigma tipico per processi cooperanti
 - processo *producer* produce informazione che è consumata dal processo *consumer* (es., *web server* produce html, *web browser* consuma)
 - Uso di buffer per produrre e consumare
 - ▶ Buffer in shared memory
 - ▶ Il produttore riempie il buffer
 - ▶ Il consumatore svuota il buffer
 - ▶ Produttore e consumatore devono sincronizzare le operazioni (mentre il produttore riempie il buffer, il consumatore lo svuota)
 - Due tipi di buffer
 - ▶ **unbounded-buffer** non c'è limite sulla dimensione del buffer (consumatore attende se vuoto)
 - ▶ **bounded-buffer** dimensione fissa del buffer (consumatore attende se vuoto, produttore attende se pieno)

Bounded-Buffer – Shared-Memory

- Un buffer limitato è condiviso tra processi in memoria condivisa

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```



- Array circolare con indici `in` (prossima free) e `out` (prima posizione full)
- Soluzione corretta, ma può allocare fino a `BUFFER_SIZE-1` items
 - Buffer pieno $((in + 1) \% BUFFER_SIZE) == out$
 - Buffer vuoto $in == out$

Bounded-Buffer – Produttore

```
item next_produced;
```

```
while (true) {
```

```
    /* produce an item in next produced */
```

```
    while (((in + 1) % BUFFER_SIZE) == out)
```

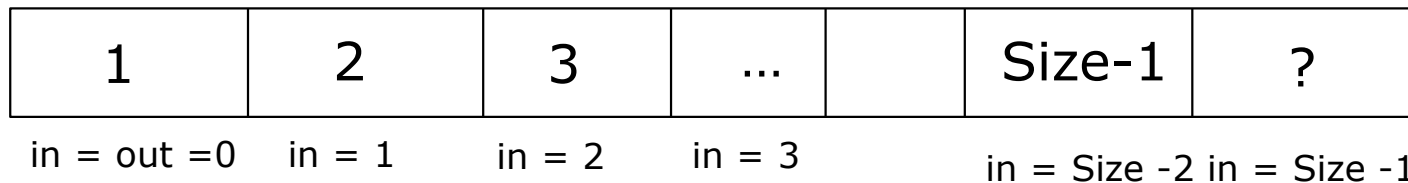
```
        ; /* do nothing */
```

```
    buffer[in] = next_produced;
```

```
    in = (in + 1) % BUFFER_SIZE;
```

```
}
```

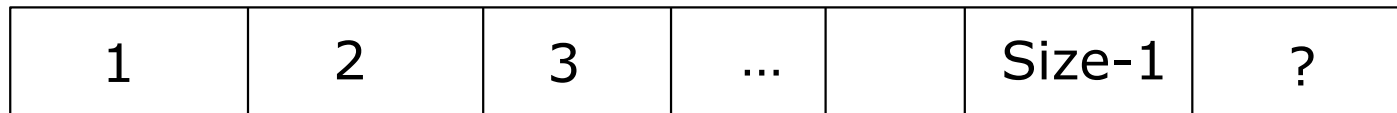
$in + 1 = \text{Size}, \text{Size} \% \text{Size} = 0 = \text{out}$



Bounded Buffer – Consumatore

```
item next_consumed;
```

```
while (true) {  
    while (in == out)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
  
    /* consume the item in next consumed */  
}
```



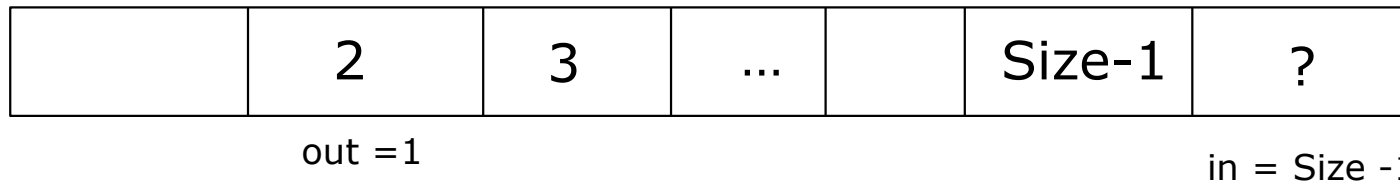
out = 0

in = Size - 1

Bounded Buffer – Consumatore

```
item next_consumed;
```

```
while (true) {  
    while (in == out)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
  
    /* consume the item in next consumed */  
}
```



Bounded-Buffer – Produttore

```
item next_produced;

while (true) {
    /* produce an item in next produced */
    while (isBufferFull)
        ; /* do nothing */
    in = (in + 1) % BUFFER_SIZE;
    buffer[in] = next_produced;
    size++;
}
```


Bounded Buffer – Consumatore

```
item next_consumed;

while (true) {
    while (isBufferEmpty)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    size--;

    /* consume the item in next consumed */
}
```